

Salute di Ferro

Dashboard sanitaria multi-ruolo - Panoramica tecnica per dev / CTO

Progetto	Salute di Ferro - piattaforma SaaS sanitaria multi-ruolo (paziente, medico, coach, admin)
Repository	monorepo: <code>salute-di-ferro</code> (web Next.js) + <code>sdf-mobile</code> (Expo / React Native)
Team	Andrea Brognera, Simone (co-founder Leone)
Consegna target	fine maggio 2026
Hosting	Vercel + Supabase (PostgreSQL gestito)

1. Sintesi

Salute di Ferro e' una piattaforma web (con companion mobile) per la gestione del percorso di cura tra pazienti, medici, coach e amministratori. Centralizza appuntamenti, dati biometrici, referti medici, terapie, diario nutrizionale e messaggistica 1:1. L'architettura e' pensata per ambito sanitario: 2FA obbligatoria per i ruoli clinici, audit log GDPR-ready, rate limiting distribuito e separazione netta dei permessi role-based.

2. Stack tecnologico

Framework & runtime

Next.js	16.2.3 (App Router, Route Handlers, Server Actions)
React	19.2.4 (+ React DOM 19.2.4)
TypeScript	5.x - strict mode
Node.js	runtime Vercel

Persistenza & auth

Database	PostgreSQL su Supabase (multi-schema: public + auth)
ORM	Prisma 7.7.0 (@prisma/adapters, @prisma/client)
Autenticazione	Supabase Auth (@supabase/ssr 0.10, @supabase/supabase-js 2.103)
2FA	Gate AAL2 forzato per DOCTOR / COACH / ADMIN (rollout via feature flag)

UI, form, state

UI kit	shadcn/ui + Base UI 1.4 + Tailwind CSS 4
Iconografia	Lucide React 1.8
Form & validazione	React Hook Form 7.72 + Zod 4.3 + @hookform/resolvers

Data fetching	TanStack Query 5.99
State client	Zustand 5.0 (store leggeri per UI state)
Grafici	Recharts 3.8
Drag & drop	@dnd-kit/core 6.3 + sortable 10.0
Tema	next-themes 0.4 (light / dark)

Integrazioni & piattaforma

Pagamenti	Stripe 22 (server) + @stripe/stripe-js 9 (client), webhook handler dedicato
Email	Resend 6.12 (transazionali)
AI	Vercel AI SDK 6 + @ai-sdk/openai 3
Rate limiting	Upstash Redis 1.37 + @upstash/ratelimit 2 (sliding window)
Feature flags	Upstash Redis (con fallback su env)
Monitoring	Sentry (@sentry/nextjs 10.49) - error tracking + health check

Tooling

- ESLint 9 + eslint-config-next 16, Prettier + prettier-plugin-tailwindcss
- tsx 4.21 per script TS (seed Prisma, utility)
- Conventional Commits, branch obbligatorio (mai lavorare su **main**)

3. Architettura

Struttura cartelle (estratto)

```
src/  
  app/  
    (auth)/      # layout group auth  
    (legal)/     # pagine legali  
    (dev)/       # debug / role bypass  
    dashboard/  
      admin/ | doctor/ | coach/ | patient/  
      messages/ | settings/  
    api/  
      cron/      # job schedulati Vercel  
      stripe/    # webhook  
      [resource]/route.ts  
  components/    # UI riusabili  
  lib/  
    auth/        # require-role, helper RBAC  
    supabase/    # client / server / middleware  
    services/    # logica di dominio  
    queries/     # query Prisma per modulo  
    email/ | calendar/ | health/  
    medical-records/ # upload referti  
    audit.ts     # audit log GDPR  
    rate-limit.ts  
    platform-settings.ts # feature flag Redis  
    prisma.ts  
  styles/ | types/
```

Pattern principali

- **App Router** con *route groups* per separare layout (auth, legal, dev, dashboard).
- **Route Handlers** (app/api/*/route.ts) per le API REST interne e i webhook.
- **Middleware globale** (src/lib/supabase/middleware.ts) per session refresh + redirect non autenticati.
- **RBAC server-side** tramite helper require-role.ts applicato in ogni page/server action protetta.
- **Cron Vercel**: reminder appuntamenti (08:00 UTC) e retention dati (04:00 UTC) configurati in vercel.json.
- **Dev bypass** (NEXT_PUBLIC_DEV_BYPASS) per impersonare ruoli in locale via query param - mai abilitato in produzione.

4. Modello dati (Prisma)

Schema PostgreSQL pensato per relazione N-a-N paziente / professionista, con stati espliciti per i percorsi di cura e tracciamento completo degli eventi.

Identita'	User, Organization (multi-tenant white-label)
Cura	CareRelationship (paziente - medico / coach, stato: ACTIVE / PAUSED / ARCHIVED)
Agenda	Appointment (IN_PERSON VIDEO_CALL), AvailabilitySlot, Appointment
Salute	BiometricLog, SymptomLog, CheckIn, MetricTarget
Terapia	TherapyItem (PRESCRIBED SELF), NutritionPlan, MealEntry
Referti	MedicalReport con 8 categorie (BLOOD_TEST, IMAGING, CARDIOLOGY, ...)
Messaging	Conversation, ConversationMember, Message
Billing	Subscription, Invoice (sincronizzate via webhook Stripe)
Audit	AuditLog (actor, action, entity, metadata, IP, UA, timestamp)
Notifiche	Notification (REMINDER / CHECK_IN / PAYMENT / SYSTEM / AI)

5. Ruoli e moduli funzionali

Paziente - /dashboard/patient

- Overview salute, tracking biometrico (peso, parametri vitali).
- Calendario appuntamenti, messaggistica con il Team di Ferro.
- Aderenza terapia, diario nutrizionale, check-in periodici.

Medico - /dashboard/doctor

- Lista pazienti + scheda clinica.
- Upload referti medici (multi-categoria) e prescrizione terapie.
- Gestione disponibilita' / slot agenda.

Coach - /dashboard/coach

- Gestione clienti, intake terapeutico, piani nutrizionali.
- Review check-in (stato PENDING / REVIEWED), target metrici.

Admin - /dashboard/admin

- Health check di sistema (Sentry + DB + servizi esterni).
- Toggle feature flag (Redis-backed, es. ENFORCE_2FA).
- Broadcast e provisioning staff.

6. Sicurezza e compliance

2FA obbligatoria sui ruoli clinici

Hard gate AAL2 (Supabase) per DOCTOR, COACH e ADMIN. Rollout progressivo via feature flag su Redis con fallback su variabile d'ambiente. Sono previste solo escape route esplicite (/dashboard/settings/security, /auth, /api) per evitare lockout durante setup.

Audit log (GDPR Art. 30)

La tabella **AuditLog** registra attore, azione (enum tipizzata), entita' coinvolta, metadata in JSON, IP e user-agent. La scrittura e' best-effort: un fallimento di logging non blocca la request, ma viene tracciato in Sentry. Pronto per i registri delle attivita' di trattamento richiesti dal GDPR.

Rate limiting distribuito

Implementazione sliding-window su Upstash Redis (HTTP-based, compatibile con runtime serverless Vercel). Per il dev locale c'e' un fallback in-memory con Map.

Error tracking & health

- Sentry SDK lato server e client (source map upload via auth token).
- Endpoint /api/admin/health testa connessione DB, Sentry, servizi esterni.

Sicurezza DB

- Multi-schema PostgreSQL (public + auth) con foreign key verso auth.users Supabase.
- Adapter Prisma su pg ufficiale - connessione pooled.
- RLS (Row Level Security) gestita lato Supabase per le tabelle critiche.

7. DevOps & deploy

Hosting

Deploy su Vercel (configurazione vercel.json). Database PostgreSQL e Auth su Supabase. Redis su Upstash. Email via Resend. Pagamenti via Stripe.

Build pipeline

```
npm run build -> prisma migrate deploy -> prisma generate -> next build
```

Cron job (Vercel)

- /api/cron/appointment-reminders - ogni giorno 08:00 UTC.
- /api/cron/retention - ogni giorno 04:00 UTC.

Variabili d'ambiente principali

- SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY
- STRIPE_SECRET_KEY, STRIPE_PUBLISHABLE_KEY, STRIPE_WEBHOOK_SECRET
- UPSTASH_REDIS_REST_URL, UPSTASH_REDIS_REST_TOKEN
- SENTRY_AUTH_TOKEN, NEXT_PUBLIC_SENTRY_DSN
- RESEND_API_KEY, OPENAI_API_KEY

Workflow di team

- Mai lavorare direttamente su **main**. Branch obbligatori: feat/, fix/, chore/, refactor/, docs/.
- Conventional Commits con scope (auth, patient, doctor, ...).

- PR piccole (1-2 giorni di lavoro), auto-merge su CI verde, review post-merge asincrona.

8. Stato e roadmap

Stato attuale	Dashboard web operativa con i 4 ruoli, billing Stripe, audit GDPR, 2FA in rollout
Companion mobile	App Expo / React Native (<code>sdf-mobile</code>) in sviluppo parallelo
Consegna	fine maggio 2026 - priorit� assoluta alla stabilit� di <code>main</code>

Documento generato in modo automatico a partire dallo stato corrente del repository. Per dettagli aggiornati su routing, comandi e convenzioni vedere i file `CLAUDE.md` e `AGENTS.md` nella root del progetto.